

Assignment 1 — Builder Pattern

Product: HTTP Request

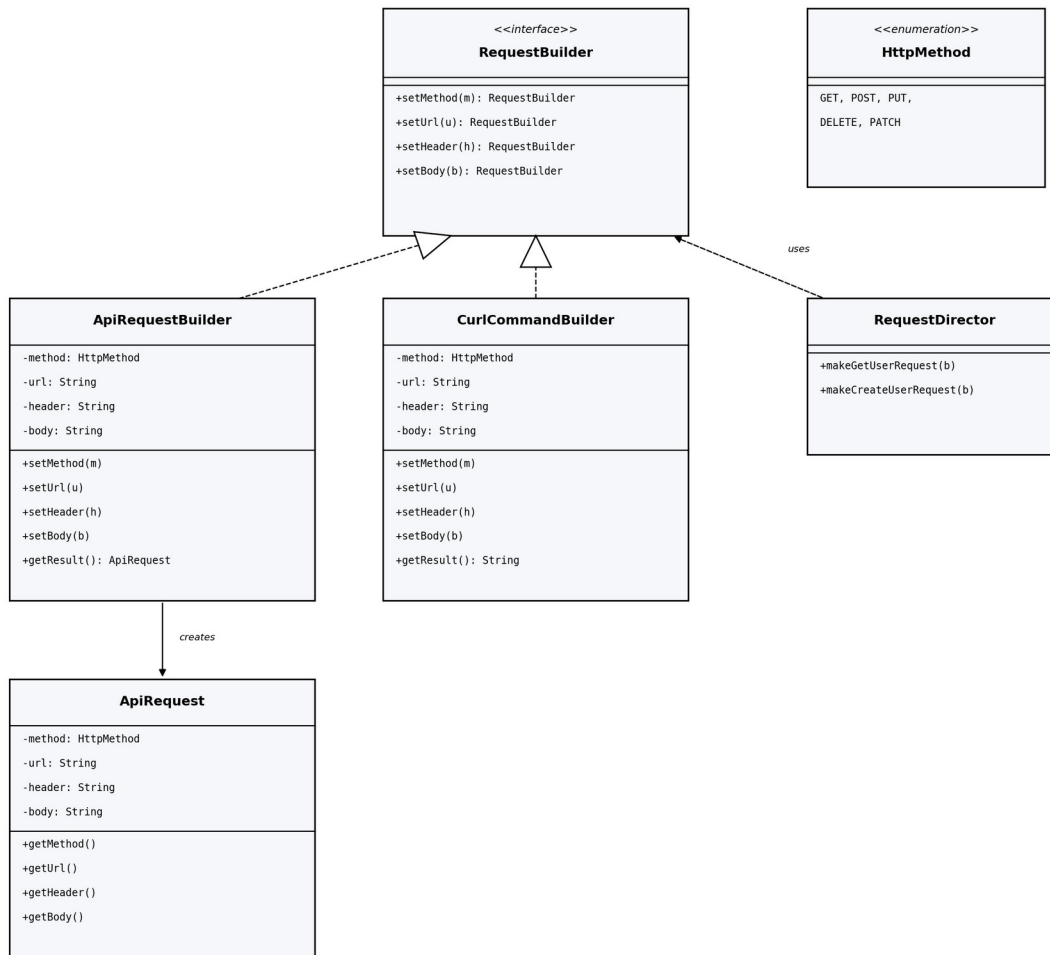
Nauryzbay Kinaiatov — Astana IT University

1. Introduction

This report describes an implementation of the Builder design pattern in Java, applied to the construction of HTTP requests. The goal of the Builder pattern is to separate the step-by-step construction of a complex object from its final representation, so that the same construction process can produce different outputs.

In this project, the same four construction steps — setting the HTTP method, URL, header, and body — are used to produce two different representations of the same request: an immutable `ApiRequest` object, and a text representation in the form of a curl command. A `RequestDirector` class encapsulates two ready-made configurations (a GET request and a POST/create-user request) and depends only on the `RequestBuilder` interface, not on any concrete builder.

2. Structure (UML Class Diagram)



RequestBuilder is an interface implemented by two concrete builders: **ApiRequestBuilder** and **CurlCommandBuilder**. Both share the same fluent setter methods, but `getResult()` is declared on each concrete builder rather than on the interface, because the two builders return different types (**ApiRequest** vs. **String**). **ApiRequestBuilder** additionally creates instances of the **ApiRequest** product. **RequestDirector** depends only on the **RequestBuilder** interface and is unaware of which concrete builder it is given, while **HttpMethod** is a supporting enum used by every class in the diagram.

3. Clean Code Principles

3.1 Meaningful, Intention-Revealing Names

Every class and method is named after what it actually does rather than using generic names. **ApiRequestBuilder** builds an **ApiRequest**; **CurlCommandBuilder** builds a curl command string; `makeGetUserRequest` and `makeCreateUserRequest` describe exactly which configuration they apply.

```
public void makeGetUserRequest(RequestBuilder builder, String url) {
```

```
builder.setMethod(HttpMethod.GET)
        .setUrl(url);
}
```

3.2 Single Responsibility Principle

Each class has exactly one reason to change. `ApiRequest` only holds data. `ApiRequestBuilder` only assembles an `ApiRequest`. `CurlCommandBuilder` only assembles a curl string. `RequestDirector` only knows reusable step sequences — it never constructs a product itself.

```
public class ApiRequest {
    private final HttpMethod method;
    private final String url;
    private final String header;
    private final String body;
    // only getters below – no construction logic, no formatting logic
}
```

3.3 No Magic Strings or Numbers

Repeated literal values are pulled out into named constants instead of being typed inline, so their meaning is explicit and they only need to change in one place.

```
private static final String FLAG_METHOD = "-X";
private static final String FLAG_HEADER = "-H";
private static final String FLAG_BODY = "-d";

private static final String JSON_HEADER = "Content-Type: application/json";
```

3.4 Immutability of the Product

`ApiRequest` cannot be modified once created: its fields are final, it has no public setters, and its constructor is package-private so that only a builder in the same package can create an instance.

```
ApiRequest(HttpMethod method, String url, String header, String body) {
    this.method = method;
    this.url = url;
    this.header = header;
    this.body = body;
}
// no setters – state cannot change after construction
```

3.5 Fail Fast with Clear Exceptions

Both concrete builders validate required fields before constructing a result and throw a descriptive exception if the state is invalid, instead of silently returning a broken object.

```
public ApiRequest getResult() {
    if (method == null || url == null || url.isBlank()) {
        throw new IllegalStateException(
            "Method and URL are required to build an ApiRequest");
    }
    return new ApiRequest(method, url, header, body);
}
```

3.6 Small, Focused Methods

Every setter method does exactly one thing — assign a field and return the builder — keeping each method short and easy to verify at a glance.

```
@Override
public ApiRequestBuilder setUrl(String url) {
    this.url = url;
    return this;
}
```

4. Conclusion

This implementation satisfies the core intent of the Builder pattern: the same ordered construction steps, driven by RequestDirector, produce two distinct representations of an HTTP request without duplicating the construction logic itself. Keeping getResult() off the shared interface allowed each builder to return its own natural type, while validation inside getResult() ensures no invalid ApiRequest or curl command can ever be produced. Overall, the design keeps the product immutable, the director decoupled from concrete builders, and the code free of magic literals or oversized constructors.

5. GitHub Repository

<https://github.com/MangoDeath/request-forge>